

Primer Avance Proyecto Integrador FleetLogix

FleetLogix - Modernización de Infraestructura de Datos

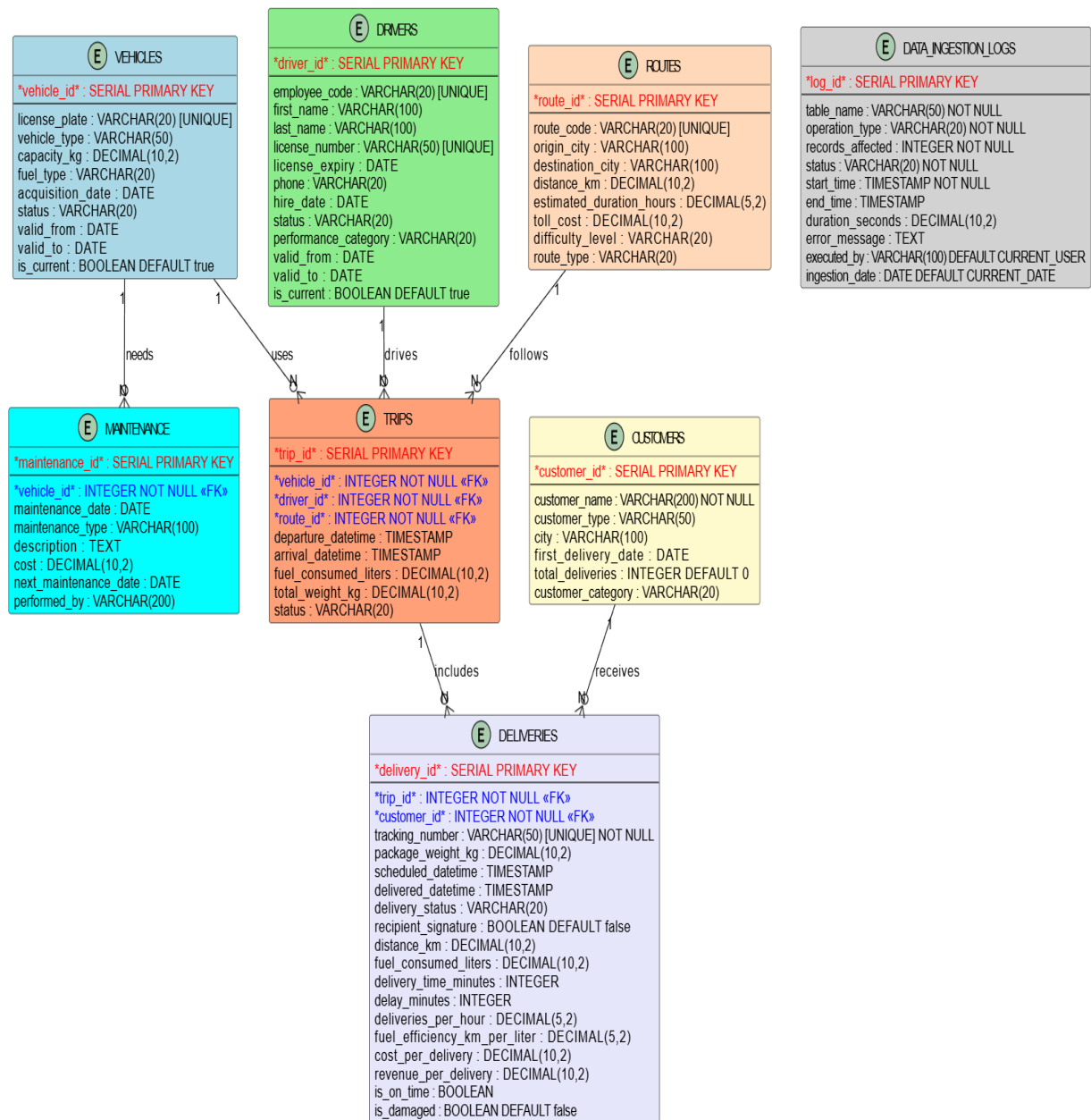
- Proyecto Integrador - Avance #1
- Autor: Matias Damian Gutierrez
- Fecha de Entrega: 15/10/2025

Introducción

Este documento presenta un análisis exhaustivo del modelo de base de datos relacional proporcionado para FleetLogix, una empresa de transporte y logística. El objetivo principal de este análisis es documentar la estructura de las tablas, las restricciones de integridad referencial (llaves foráneas) y las relaciones entre entidades, sentando las bases para la posterior generación de datos sintéticos masivos y coherentes. El análisis se realizó mediante la ejecución de consultas de inspección del sistema en la base de datos PostgreSQL y la elaboración de un diagrama Entidad-Relación (ER) que visualiza el esquema de manera profesional.

También en este documento se explica el archivo "**01_data_generation.py**" que es un código que sirve para llenar las tablas con datos sintéticos para una prueba de que el sistema funciona y se utilizara para los siguientes avances

Análisis del Modelo de Datos Relacional



DOCUMENTACIÓN DEL MODELO RELACIONAL – SISTEMA LOGÍSTICO

1. VISIÓN GENERAL DEL MODELO

El esquema SQL define un modelo relacional orientado a la gestión logística, incluyendo transporte, entregas, mantenimiento y auditoría.

Se compone de siete tablas principales, agrupadas según su función dentro del sistema.

Tipo de tabla	Tablas	Descripción general
Entidades base	vehicles, drivers, routes, customers	Representan objetos reales del negocio.
Eventos / operaciones	trips, deliveries, maintenance	Registros operativos o transaccionales.
Auditoría	data_ingestion_logs	Bitácora de operaciones o cargas de datos.

2. RELACIONES ENTRE TABLAS

El siguiente mapa relacional resume cómo interactúan las principales entidades del sistema:

vehicles → trips → deliveries → customers

drivers → trips

routes → trips

vehicles → maintenance

Constraints de Integridad Referencial:

-- Trips constraints

```
ALTER TABLE trips ADD CONSTRAINT fk_trips_vehicle FOREIGN KEY  
(vehicle_id) REFERENCES vehicles(vehicle_id);
```

```
ALTER TABLE trips ADD CONSTRAINT fk_trips_driver FOREIGN KEY  
(driver_id) REFERENCES drivers(driver_id);
```

```
ALTER TABLE trips ADD CONSTRAINT fk_trips_route FOREIGN KEY (route_id)  
REFERENCES routes(route_id);
```

-- Deliveries constraints

```
ALTER TABLE deliveries ADD CONSTRAINT fk_deliveries_trip FOREIGN KEY  
(trip_id) REFERENCES trips(trip_id);
```

```
ALTER TABLE deliveries ADD CONSTRAINT fk_deliveries_customer FOREIGN  
KEY (customer_id) REFERENCES customers(customer_id);
```

-- Maintenance constraints

```
ALTER TABLE maintenance ADD CONSTRAINT fk_maintenance_vehicle FOREIGN  
KEY (vehicle_id) REFERENCES vehicles(vehicle_id);
```

Constraints de Unicidad:

- vehicles.license_plate UNIQUE
- drivers.employee_code UNIQUE
- drivers.license_number UNIQUE
- routes.route_code UNIQUE

- `deliveries.tracking_number` UNIQUE

Implementación de SCD Type 2

El modelo implementa Slowly Changing Dimensions Type 2 en las tablas `vehicles` y `drivers`:

```
-- Estructura SCD Type 2
valid_from DATE,      -- Fecha de inicio de vigencia
valid_to DATE,        -- Fecha de fin de vigencia (NULL para actual)
is_current BOOLEAN    -- Indicador de versión actual
```

Ventajas para el Data Warehouse:

- Permite tracking histórico de cambios
- Facilita análisis temporales
- Compatible con herramientas de BI

3. DESCRIPCIÓN DE LAS TABLAS Y RELACIONES

- `vehicles`: Contiene los vehículos utilizados. Se relaciona con `trips` y `maintenance` (1:N).
- `drivers`: Registra conductores. Relación 1:N con `trips`.
- `routes`: Define rutas disponibles. Relación 1:N con `trips`.
- `customers`: Representa clientes que reciben entregas. Relación 1:N con `deliveries`.
- `trips`: Registra viajes, relacionándose con `vehicles`, `drivers` y `routes`. Relación 1:N con `deliveries`.
- `deliveries`: Guarda entregas, vinculadas a `trips` y `customers`.
- `maintenance`: Controla mantenimientos por vehículo (1:N con `vehicles`).
- `data_ingestion_logs`: Tabla de auditoría sin relaciones directas.

4. FLUJO DE DATOS EN EL SISTEMA

1. Se definen los recursos base: vehículos, conductores, rutas y clientes.
2. Se planifica un viaje (`trips`) asignando vehículo, conductor y ruta.
3. Se registran entregas (`deliveries`) vinculadas a viajes y clientes.
4. Se registran mantenimientos (`maintenance`) por vehículo.
5. Se monitorean las operaciones mediante la tabla de auditoría (`data_ingestion_logs`).

5. TIPOS DE RELACIONES

Relación	Tipo	Descripción
----------	------	-------------

vehicles → trips	1:N	Un vehículo puede tener múltiples viajes.
drivers → trips	1:N	Un conductor puede realizar varios viajes.
routes → trips	1:N	Una ruta puede asociarse a distintos viajes.
trips → deliveries	1:N	Un viaje puede incluir varias entregas.
customers → deliveries	1:N	Un cliente puede recibir múltiples entregas.
vehicles → maintenance	1:N	Un vehículo puede tener varios mantenimientos.

Explicación de el código de “Data_Generation”

Arquitectura del Script Python

Configuración y Logging:

```
# Configuración de logging y variables de entorno
load_dotenv() # Carga variables desde .env
logging.basicConfig(...) # Sistema de logs con archivo y consola

# Semillas para reproducibilidad
fake = Faker("es_ES") # Datos en español
random.seed(42) # Resultados consistentes
```

Estrategia de Generación:

- Semillas reproducibles: random.seed(42), np.random.seed(42)
- Localización española: Faker("es_ES")
- Inserción masiva: executemany() para performance

Propósito: Configura el entorno, logging y asegura que los datos generados sean consistentes entre ejecuciones.

CONEXIÓN A BASE DE DATOS

```
def get_db_connection():
    conn = psycopg2.connect(
        dbname=os.getenv('DB_NAME'), # Desde variables de entorno
        user=os.getenv('DB_USER'),
        password=os.getenv('DB_PASSWORD'),
        host=os.getenv('DB_HOST'),
        port=os.getenv('DB_PORT')
    )
```

Propósito: Establece conexión segura con PostgreSQL usando credenciales externas.

CONFIGURACIÓN DIMENSIONAL (Para Data Warehouse)

```
def get_dimensional_config():
    return {
        'performance_categories': ['Alto', 'Medio', 'Bajo'],
        'customer_categories': ['Premium', 'Regular', 'Ocasional'],
        'difficulty_levels': ['Fácil', 'Medio', 'Difícil']
    }
```

Propósito: Define categorías consistentes para el modelo dimensional de Snowflake.

MATRIZ DE DISTANCIA REALISTA

```
def get_consistent_distances():
    distance_matrix = {
        ('Buenos Aires', 'Córdoba'): (700, 'Medio', 'Interurbana'),
        ('Buenos Aires', 'Rosario'): (300, 'Fácil', 'Interurbana'),
        # ... más rutas con (distancia, dificultad, tipo)
    }
```

Propósito: Mantiene coherencia en distancias y características de rutas.

SISTEMA DE LOGGING EN BD

```
def log_to_database(conn, table_name, operation_type, records_affected, status, start_time, end_time=None, error_message=None):
    # Inserta registro de auditoría en tabla data_ingestion_logs
```

Propósito: Registra cada operación ETL para trazabilidad y monitoreo.

INSERCIÓN MASIVA OPTIMIZADA

```
def insert_data_massive(cur, data_tuples, columns, table_name):
    placeholders = ', '.join(['%s'] * len(columns))
    query = f"INSERT INTO {table_name} ({columns_str}) VALUES ({placeholders})"
    cur.executemany(query, data_tuples) # Inserción eficiente
```

Propósito: Inserta grandes volúmenes de datos de manera eficiente usando executemany.

GENERACIÓN DE VEHÍCULOS (SCD Type 2)

```
def generate_vehicles():
    vehicles_data.append((
        fake.unique.license_plate(), # Patente única
        random.choice(vehicle_types), # Tipo de vehículo
        round(random.uniform(200, 20000), 2), # Capacidad
        # Campos SCD Type 2:
        acquisition_date, # valid_from
        None, # valid_to (null = versión actual)
        True # is_current
    ))
```

Propósito: Genera vehículos con soporte para cambios históricos (Slowly Changing Dimensions Type 2).

Slowly Changing Dimensions (SCD) Type 2

Definición

El SCD Type 2 es una técnica de modelado dimensional que preserva el historial completo de cambios en los datos maestros, creando nuevos registros cada vez que ocurre una modificación en los atributos de una dimensión.

GENERACIÓN DE CONDUCTORES

```
def generate_drivers():
    # Garantiza códigos únicos
    while True:
        emp_code = f"EMP{random.randint(100, 999)}"
        if emp_code not in used_emp_codes:
            used_emp_codes.add(emp_code)
            break
```

Propósito: Crea conductores con identificadores únicos y categorías de performance.

GENERACIÓN DE RUTAS

```
def generate_routes():
    # Usa la matriz de distancias para consistencia
    base_distance, difficulty, route_type = distance_matrix.get((origin, destination))
    final_distance = round(base_distance * random.uniform(0.98, 1.02), 2)
```

Propósito: Genera rutas con distancias realistas y atributos para análisis dimensional.

GENERACIÓN DE VIAJES INTELIGENTE

```
def generate_trips(n_trips, start_date, end_date, vehicle_ids, driver_ids, routes_df, hourly_dist=None):
    # Distribución horaria realista
    hourly_probs = get_hourly_distribution(peaks=[8, 9, 17]) # Horas pico

    # Asignación aleatoria pero controlada
    chosen_routes = np.random.choice(route_ids, size=n_trips)
    chosen_vehicles = np.random.choice(vehicle_ids, size=n_trips)
```

Propósito: Crea viajes con distribución temporal realista y relaciones válidas entre entidades.

GENERACIÓN DE ENTREGAS CON MÉTRICAS

```
def generate_and_insert_deliveries():
    # Calcula métricas para análisis
    delay_minutes = max(0, random.randint(-10, 45))
    is_on_time = delay_minutes <= 15
    deliveries_per_hour = round(60 / delivery_time_minutes, 2)
    fuel_efficiency = round(distance_for_delivery / fuel_for_delivery, 2)
```

Propósito: Genera entregas con métricas calculadas para la tabla de hechos fact_deliveries

FUNCIÓN PRINCIPAL ORQUESTADORA

```
def main():
    # Orden específico por dependencias
    generate_vehicles()      # Primero: dimensión independiente
    generate_drivers()      # Segundo: otra dimensión
    generate_routes()       # Tercero: dimensión de rutas
    generate_customers()    # Cuarto: dimensión de clientes
    generate_and_insert_trips() # Quinto: hechos que usan dimensiones
    generate_and_insert_deliveries() # Sexto: más hechos
    generate_and_insert_maintenance() # Séptimo: datos adicionales
```

Propósito: Coordina la generación en el orden correcto respetando dependencias referenciales.

CARACTERÍSTICAS CLAVE DEL CÓDIGO:

1. Reproducibilidad: Semillas fijas para datos consistentes
2. Escalabilidad: Inserción masiva para grandes volúmenes
3. Calidad de datos: Validaciones y consistencia
4. Preparado para DW: Estructura optimizada para modelo estrella
5. Auditoría: Logging completo y trazabilidad
6. Manejo de errores: Excepciones controladas y rollback

El código genera un ecosistema completo de datos de logística listo para análisis en un data warehouse.

Explicacion de generate_and_insert_trips y get_hourly_distribution

Generacion de rutas:

```
def generate_and_insert_trips():
    """Genera e inserta datos de trips optimizados para el proceso
    ETL."""

    # Obtiene datos existentes para relaciones
    cur.execute("SELECT vehicle_id FROM vehicles")
    vehicle_ids = [row[0] for row in cur.fetchall()]

    cur.execute("SELECT driver_id FROM drivers")
    driver_ids = [row[0] for row in cur.fetchall()]

    # Configurar distribución horaria realista
    hourly_dist = get_hourly_distribution(peaks=[8, 9, 17],
    peak_weights=[4, 3, 5], spread=1)

    # Llama a la función que genera los trips
    df_trips = generate_trips(
        n_trips=100000,
        start_date='2023-01-01',
        end_date=datetime.now().date(),
        vehicle_ids=vehicle_ids,
        driver_ids=driver_ids,
        routes_df=routes_df,
        hourly_dist=hourly_dist,
        seed=42
    )
```

Flujo Completo del Proceso:

1. Prepara referencias → Obtiene IDs válidos de BD
2. Configura patrones temporales → Distribución realista de horas pico

3. Genera datos sintéticos → 100K viajes con:
 - Fechas entre 2023-01-01 y hoy
 - Relaciones válidas (vehículo → conductor → ruta)
 - Distribución temporal realista
 - Resultados reproducibles (seed 42)

Generacion de horarios distribuidos:

```
def get_hourly_distribution(peaks=None, peak_weights=None, spread=1):
    """
    Genera distribución de probabilidades para las horas del día.
    Optimizada para análisis temporal en dim_time.

    Args:
        peaks: Lista de horas pico [8, 9, 17] = horas de mayor
actividad
        peak_weights: Pesos para las horas pico [4, 3, 5] = intensidad
        spread: Factor de dispersión (afecta horas alrededor del pico)

    Returns:
        probs: Array de probabilidades para cada hora (0-23)
    """
    w = np.ones(24, dtype=float) # Base: todas las horas iguales

    # Aumentar probabilidad en horas pico
    if peaks:
        if peak_weights is None:
            peak_weights = [3] * len(peaks)
        for h, wt in zip(peaks, peak_weights):
            h = int(h) % 24
            w[h] += wt # Aumenta peso en hora pico

    # Suavizar la distribución
    if spread > 0:
        kernel = np.ones(2 * spread + 1, dtype=float)
        w = np.convolve(w, kernel, mode='same')

    probs = w / w.sum() # Normalizar a probabilidades
    return probs
```

Propósito

Genera una distribución de probabilidades para las 24 horas del día, optimizada para análisis temporal en procesos ETL y dimensiones de tiempo.

- Crea un array de 24 posiciones (0-23 horas)
- Con pesos: Aplica los pesos personalizados
- Ejemplo: peaks=[8, 9, 17] con peak_weights=[4, 3, 5]:
 - Hora 8: peso base 1 + 4 = 5
 - Hora 9: peso base 1 + 3 = 4

- Hora 17: peso base $1 + 5 = 6$
- Convolución: Propaga influencia de horas pico a horas adyacentes
- Resultado: Distribución más realista y continua

Conclusión del Avance #1

El presente documento ha establecido las bases fundamentales para el proyecto de modernización de infraestructura de datos de FleetLogix. A través del análisis exhaustivo del modelo relacional existente, se ha logrado:

Logros Alcanzados

1. Documentación Completa del Esquema: Se ha elaborado una documentación detallada de todas las tablas, relaciones y restricciones del modelo actual, identificando claramente las entidades base, operacionales y de auditoría.
2. Arquitectura de Datos Clarificada: El análisis de relaciones y flujos de datos proporciona una comprensión profunda de cómo interactúan los diferentes componentes del sistema logístico.
3. Preparación para Data Warehouse: La identificación de estructuras SCD Type 2 en vehículos y conductores demuestra que el modelo está preparado para implementaciones avanzadas de business intelligence.
4. Herramientas de Generación Desarrolladas: El código `01_data_generation.py` representa una solución robusta y escalable para la creación de datos sintéticos, incorporando mejores prácticas como:
 - Reproducibilidad mediante semillas fijas
 - Generación masiva optimizada
 - Consistencia referencial garantizada
 - Distribuciones temporales realistas
 - Preparación para modelo dimensional